



Chap1 Programming & Python

第1章 程序设计和Python语言

- **教材:**

- 《Python程序设计》，张莉主编，高等教育出版社，2022年第二版
- 《Python程序设计实验指导》，张莉、陶烨编，高等教育出版社，2022年第一版

- **课程网站:**

- <https://yiqinnju.github.io/course/python/python.html>

- **联系信息:**

- yiqincs at nju.edu.cn 仙林校区计算机系楼501室

课程规则

3

- 成绩组成：**平时40%+期末60%**
- 平时成绩：
 - **课后作业15%**：在课后布置的上机作业，主要完成方式是通过OJ系统提交自己的程序代码。
 - **课堂小测15%**：在某两次上机课（具体时间会提前通知）上会进行的闭卷测试，主要基于考试系统完成一系列程序阅读题、程序填空题和程序改错题。
 - **课程项目10%**：在学期后半段布置，会给定主题范围，自由命题，个人组队。需要提交一个文档说明项目的主要功能、代码结构和主要运行结果截图。
- 期末考试：
 - 闭卷测试，基于考试系统（和课堂小测先类似）完成一系列程序阅读题、程序填空题、程序改错题和编程题。

1.1 Programming Overview

1.1 程序设计概述

程序 Program

5

- 什么是程序?
 - 程序 = 算法 + 数据结构
 - 数据结构：程序处理的对象
 - 算法：程序处理的过程
- 程序 (Program) 和软件 (Software) / 系统 (System) 的区别
 - 软件 = 程序 + 文档
 - 系统 = 多个软件组成的聚合体

程序设计 Programming

6

- 什么是程序设计：如何编写一个“好”的程序
 - 面向业务问题的算法设计能力
 - 面向业务场景的工程实践能力
 - 面向AI辅助的系统验证能力
- “好”的程序是什么？
 - 功能性方面：正确性，安全性
 - 非功能性方面：高效性，速度快、体积小、可靠性高

排序任务

7

- 把五位同学的成绩按照从低到高排列：
 - 排序前：82 65 91 76 88
 - 排序后：65 76 82 88 91
- 排序算法：排序的过程是怎样的？
- 排序程序：将排序过程“翻译”为计算机能够理解并执行的语言
- 排序软件：将排序程序封装为别人能够使用的产品
- 排序系统：为排序软件提供持续性的服务

排序算法

8

- 每轮选出剩余成绩中的最小值，放到结果末尾，再移走它
 - 说明“怎么做”，可以用自然语言、流程图或伪代码表达

步骤	剩余成绩	已排好的结果
开始	82, 65, 91, 76, 88	空
第 1 轮	82, 91, 76, 88	65
第 2 轮	82, 91, 88	65, 76
第 3 轮	91, 88	65, 76, 82
继续到结束	空	65, 76, 82, 88, 91

排序程序

9

- 选择排序的Python程序:

```
scores = [82, 65, 91, 76, 88]
```

```
result = []
```

```
while scores:
```

```
    smallest = min(scores)
```

```
    result.append(smallest)
```

```
    scores.remove(smallest)
```

```
print(result)
```

排序软件：程序与配套规约、文档

10

- 如何将排序程序作为一个工具交给同学使用？
 - 规约：实现约定的软件行为
 - 文档：帮助用户使用和维护软件的文件

输入：含姓名、学号和成绩的表格

规则：升序排列，同分保留原顺序

输出：完整记录，不能错配姓名

异常：成绩缺失时提示具体行号

使用说明：如何导入与导出

格式说明：列名和成绩范围

测试记录：同分、空表等情形

维护说明：版本和修改记录

排序系统

11

- 如何为全校教师在成绩管理系统中提供排序服务？

关注点	具体问题	可能的应对方式
可靠性	会不会漏人、错配成绩？	校验记录，完整保存结果
可用性	需要时能用吗？故障后多久恢复？	故障监测，备份与恢复
效率	数据多、同时使用的人多，会慢吗？	改进算法、读写和资源安排

计算机世界的四个层次

12

层次	核心问题	排序案例中的体现
算法	怎样解决问题？	反复选择剩余成绩的最小值
程序	怎样让计算机执行？	用 Python 写出循环与操作
软件	怎样交付、使用和维护？	程序及配套规约、数据和文档
系统	怎样在环境中持续运行？	软件与设备、网络、人员流程协作

LLM时代的软件开发

13

- 从完成任务的承包者到提出任务并进行验收的发包人
 - 提出问题：把任务说明清楚
 - 验证回答：用证据检查结果

目标： 给谁用，要解决什么问题

背景： 提供数据、代码与运行环境

约束： 说明规则、异常和性能要求

交付： 明确需要方案、代码或文档

设计： 检查逻辑与假设是否合理

程序： 实测同分、空表和缺失值

软件： 对照规约检查数据与文档

系统： 实测并发、响应与故障恢复

1.2 Python Introduction

1.2 PYTHON简介

编程语言流行指数——TIOBE

15

Aug 2026	Aug 2025	Change	Programming Language		Ratings	Change
1	1			Python	18.53%	-7.61%
2	3	⬆		C	11.10%	+2.07%
3	2	⬇		C++	8.62%	-0.56%
4	4			Java	8.25%	-0.34%
5	5			C#	4.09%	-1.43%
6	6			JavaScript	2.63%	-0.52%
7	7			Visual Basic	2.18%	-0.15%
8	12	⬆		SQL	1.88%	+0.16%
9	14	⬆		R	1.56%	+0.19%
10	18	⬆		Rust	1.45%	+0.31%

<https://www.tiobe.com/tiobe-index/>

Python应用实例

16



1.2.1 PYTHON的历史和特性

Python的诞生

18



Guido van Rossum

- 第1个Python编译器/解释器于1991年诞生
- Python名称来自Guido挚爱的电视剧Monty Python's Flying Circus
- Python介于C和Shell之间、功能全面、易学易用、可扩展

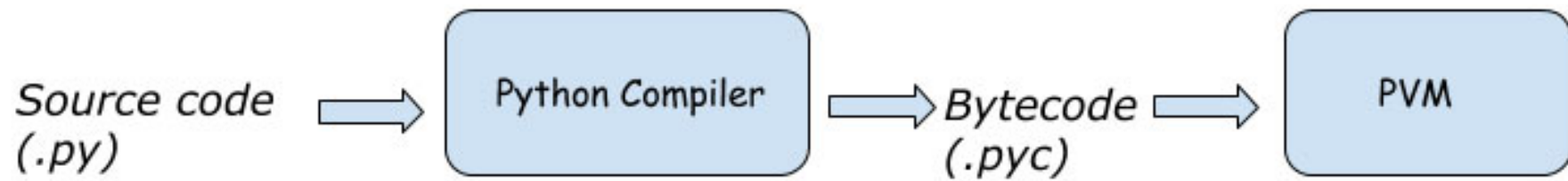
Python的哲学

19



Python的运行机制

20



- **胶水语言**

- 很容易和其他著名的程序语言连接（C/C++），集成封装

- **脚本语言**

- 高级脚本语言，比脚本语言只能处理简单任务强大

- **面向对象语言**

- 完全支持继承、重载、派生、多继承

Python的应用（一）

22

Web
开发

Python定义了WSGI标准应用接口来协调http服务器与基于Python的Web程序之间的沟通

Python提供各种库，具有极其便捷强大的数据处理和统计功能

大数据

Python的应用（二）

23

人工
智能

基于丰富的Python第三方库可以便捷高效地实现人工智能的各个阶段任务

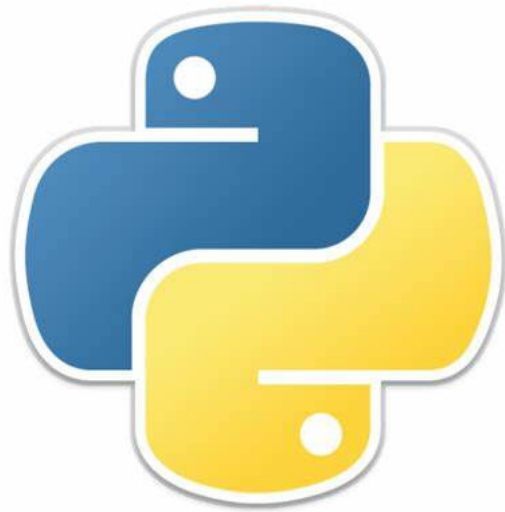
可用于计算机游戏三维场景制作和各种专业领域应用

多媒体

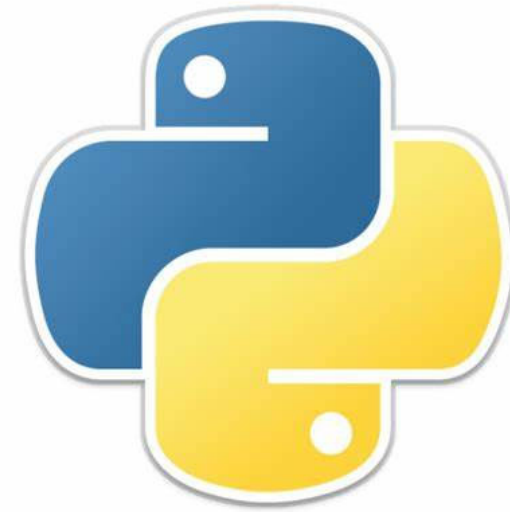
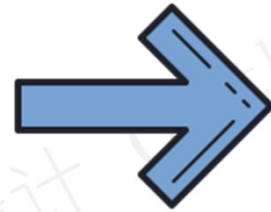
1.2.2 PYTHON的版本

Python的2个版本

25



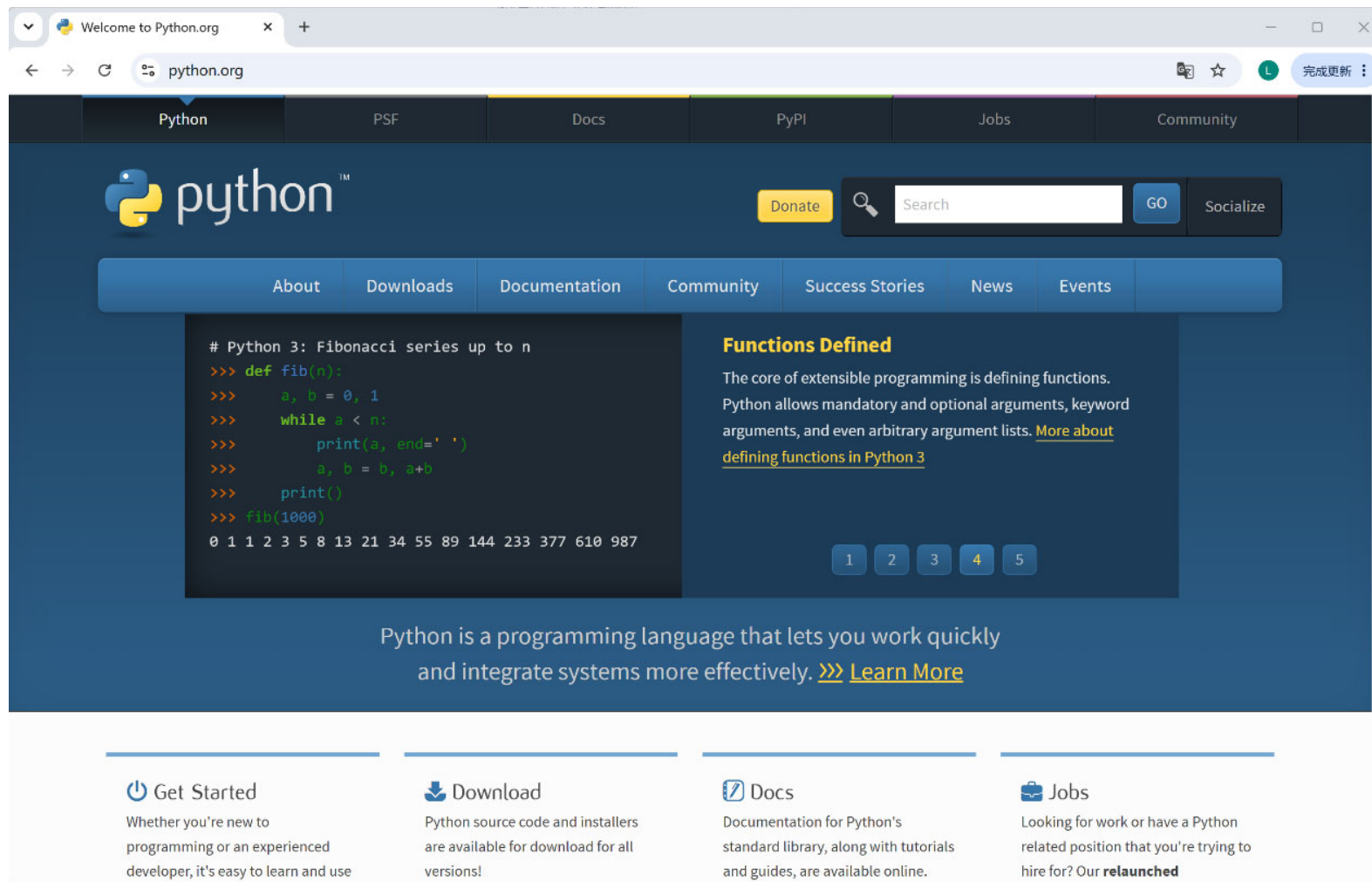
Python 2.x



Python 3.x

Python官方网站

26





The Zen of Python

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

```
>>> import this
```

by Tim Peters

1.3 Python Development Environment and Runtime

1.3 PYTHON开发环境和运行方式

经典 Hello World

29



```
string = 'Hello, World!'
print(string)
```

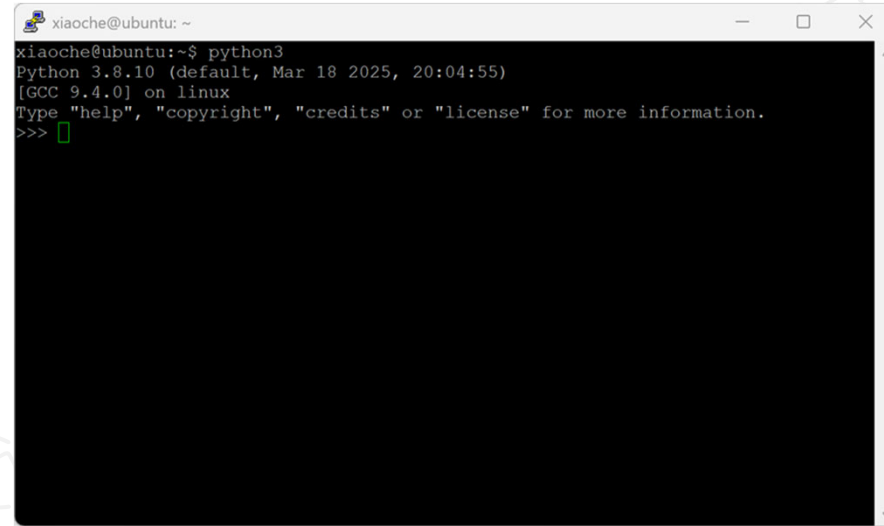
南京入

1.3.1 PYTHON开发环境

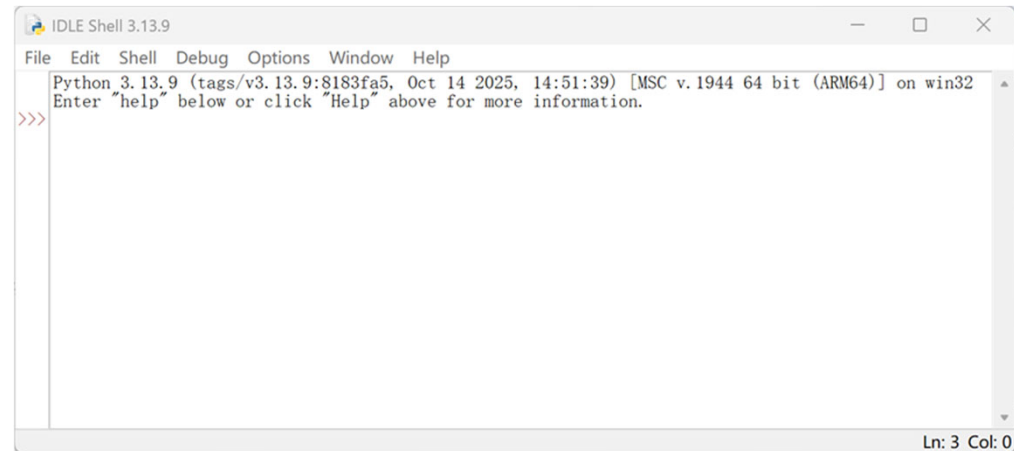
Python开发环境1

31

- **Mac OS & Linux**
 - \$ python
 - \$ python3
- **Python内置IDE**
 - IDLE



```
xiaoche@ubuntu: ~  
xiaoche@ubuntu:~$ python3  
Python 3.8.10 (default, Mar 18 2025, 20:04:55)  
[GCC 9.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```



```
IDLE Shell 3.13.9  
File Edit Shell Debug Options Window Help  
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:51:39) [MSC v.1944 64 bit (ARM64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>>
```

Python开发环境2

32



最便捷方案：Python平台Anaconda

33

• Anaconda

– download the installer

官网/国内清华或南大等镜像站

<https://www.anaconda.com/products/individual>

<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/>

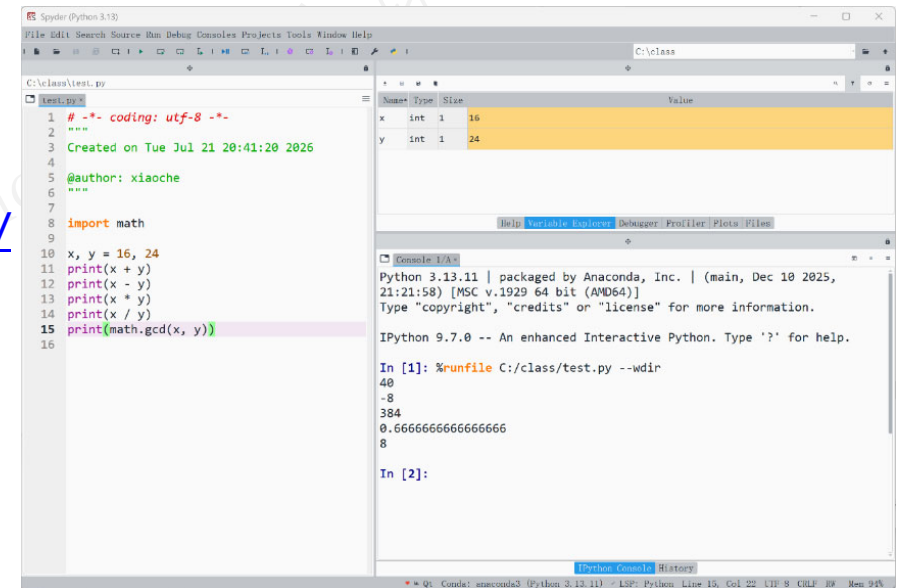
<https://mirrors.nju.edu.cn/anaconda/>

– install and use

勾选 “Add Anaconda to my PATH environment variable” (如果有)

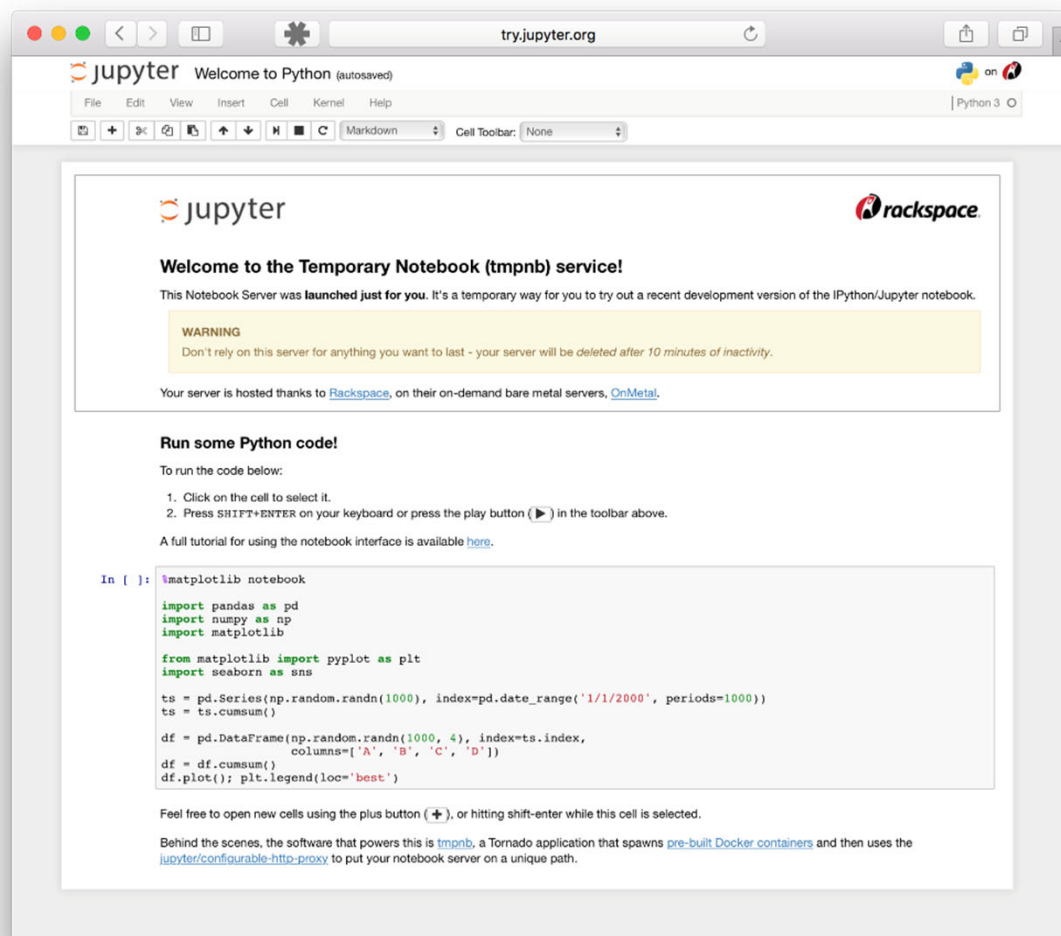
注意安装路径中不能有**中文**

安装后一般使用Spyder解释器编辑和执行程序



Jupyter Notebook

34

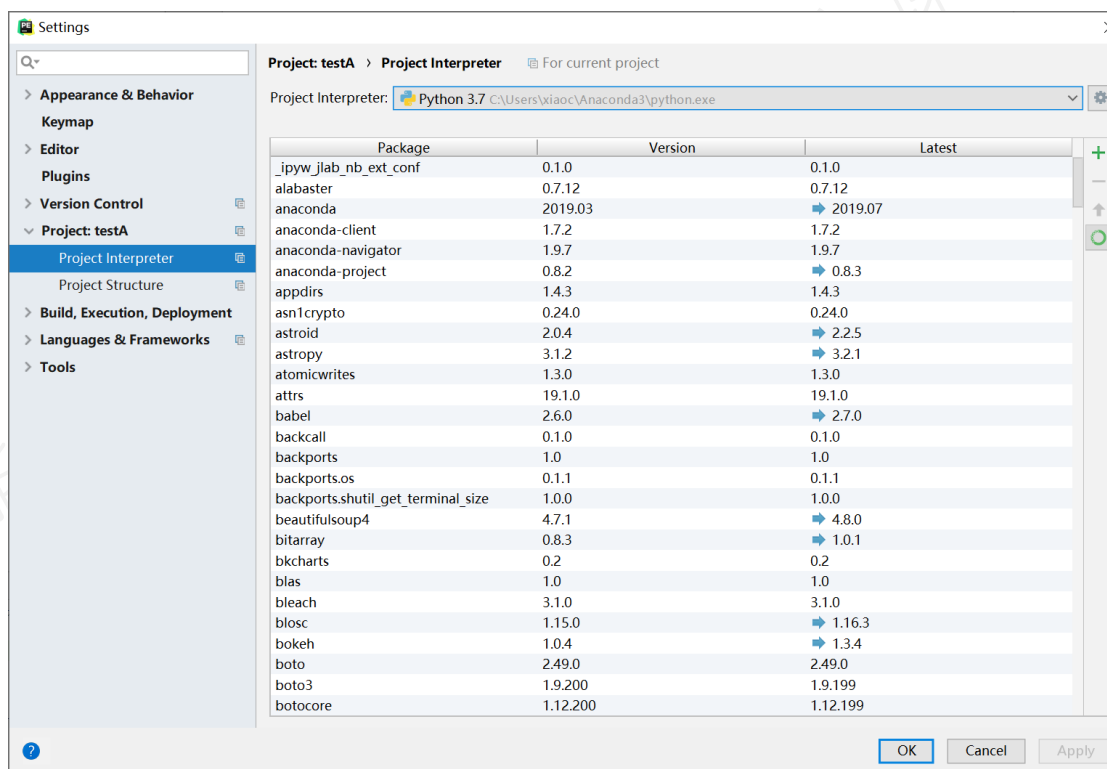


进阶推荐（稳定+丰富的第三方包）

35

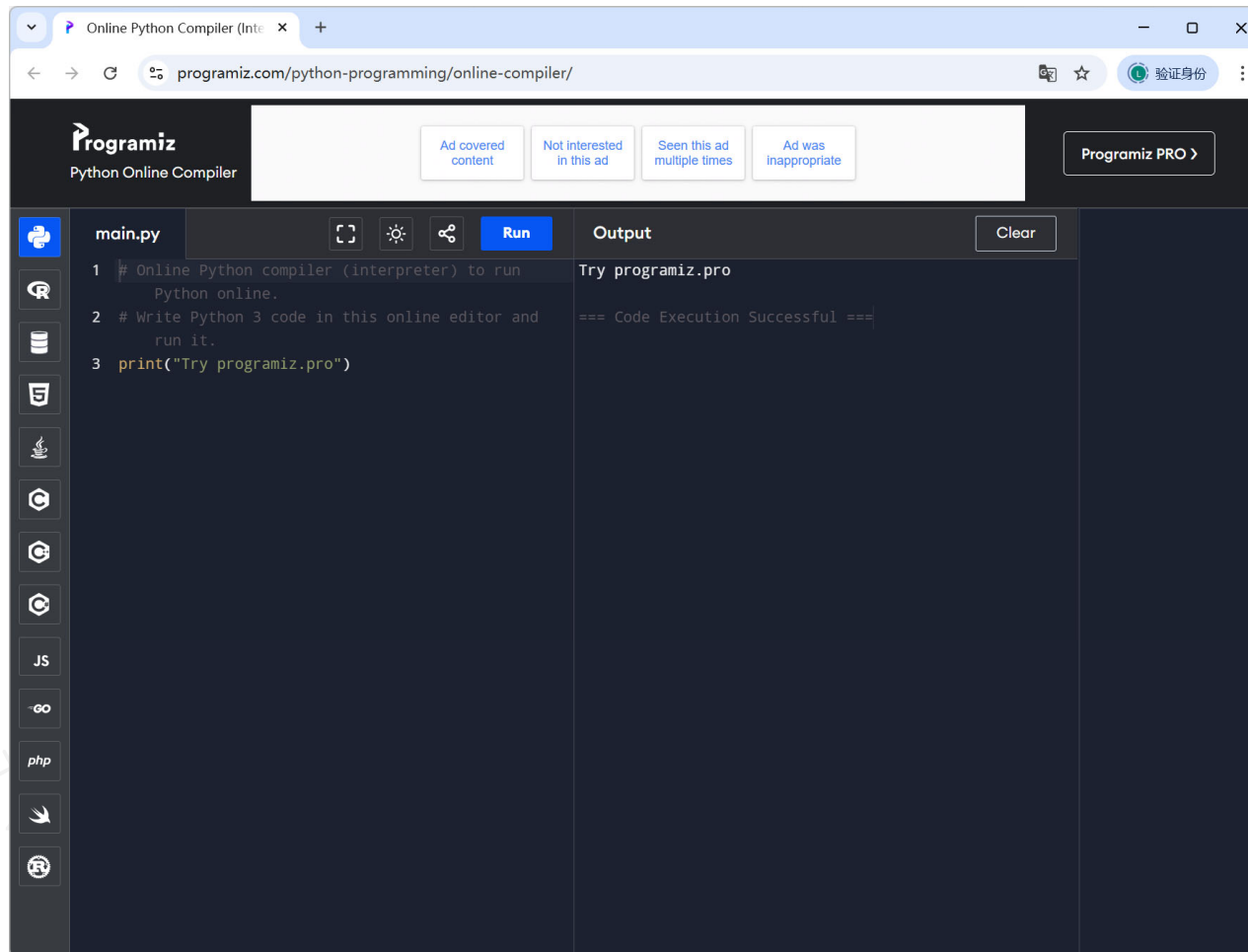
PyCharm/VS Code + Anaconda interpreter

使用“Existing interpreter”，选择Anaconda的python.exe路径



WebIDE

36



<https://www.programiz.com/python-programming/online-compiler/>

Install/Uninstall packages

37

- 用pip/conda/uv等在操作系统终端（如anaconda prompt）安装/更新/删除第三方包（官方包仓库地址 <https://pypi.org/simple>, <https://repo.anaconda.com/>)
\$ pip install 包名
\$ pip install 包名 -i https://pypi.tuna.tsinghua.edu.cn/simple
\$ pip install 包名==版本号
\$ pip install --upgrade 包名
\$ pip uninstall 包名
\$ pip list

补充：利用包管理工具venv/conda创建虚拟环境

38

```
$ conda create -n myenv python=版本号
```

```
$ conda activate myenv
```

```
$ conda install numpy/pandas
```

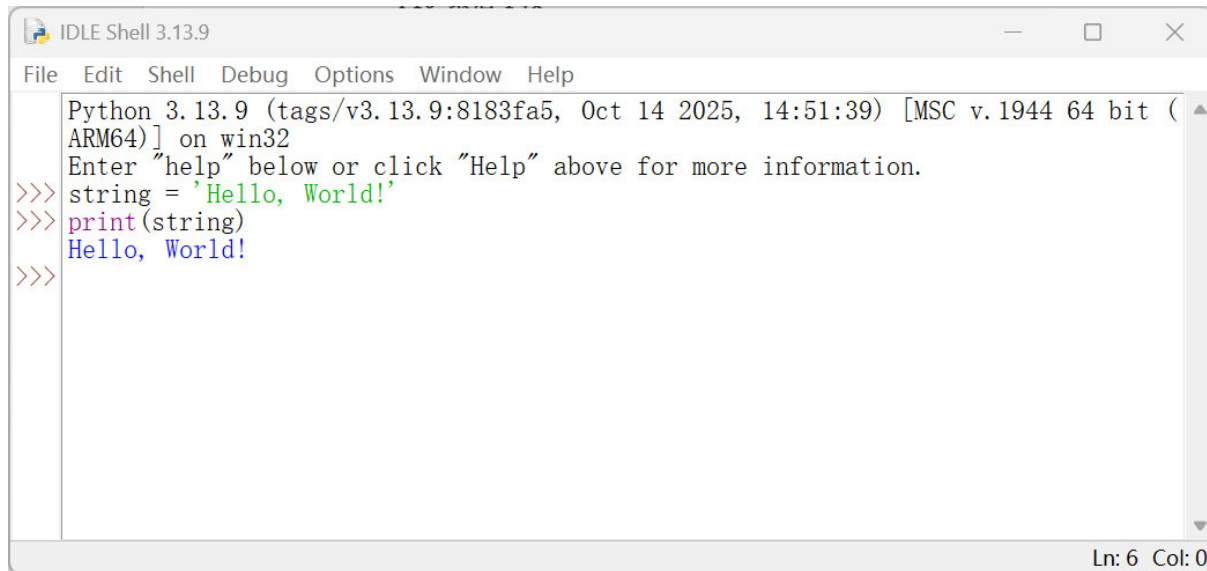
- 虚拟环境是一种用于隔离不同Python项目所需依赖的机制，它允许在同一台机器上同时安装和管理多个Python版本，以及每个版本所需的包
- 通过使用虚拟环境，可以避免不同项目之间的依赖冲突，并且可针对特定项目选择特定的Python版本

1.3.2 PYTHON运行方式

Python的运行方式（一）

40

Shell方式



The screenshot shows the IDLE Shell 3.13.9 window. The title bar reads 'IDLE Shell 3.13.9'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area contains the following text: 'Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:51:39) [MSC v.1944 64 bit (ARM64)] on win32', 'Enter "help" below or click "Help" above for more information.', and a Python prompt '>>>' followed by the code 'string = 'Hello, World!'', 'print(string)', and the output 'Hello, World!'. The status bar at the bottom right shows 'Ln: 6 Col: 0'.

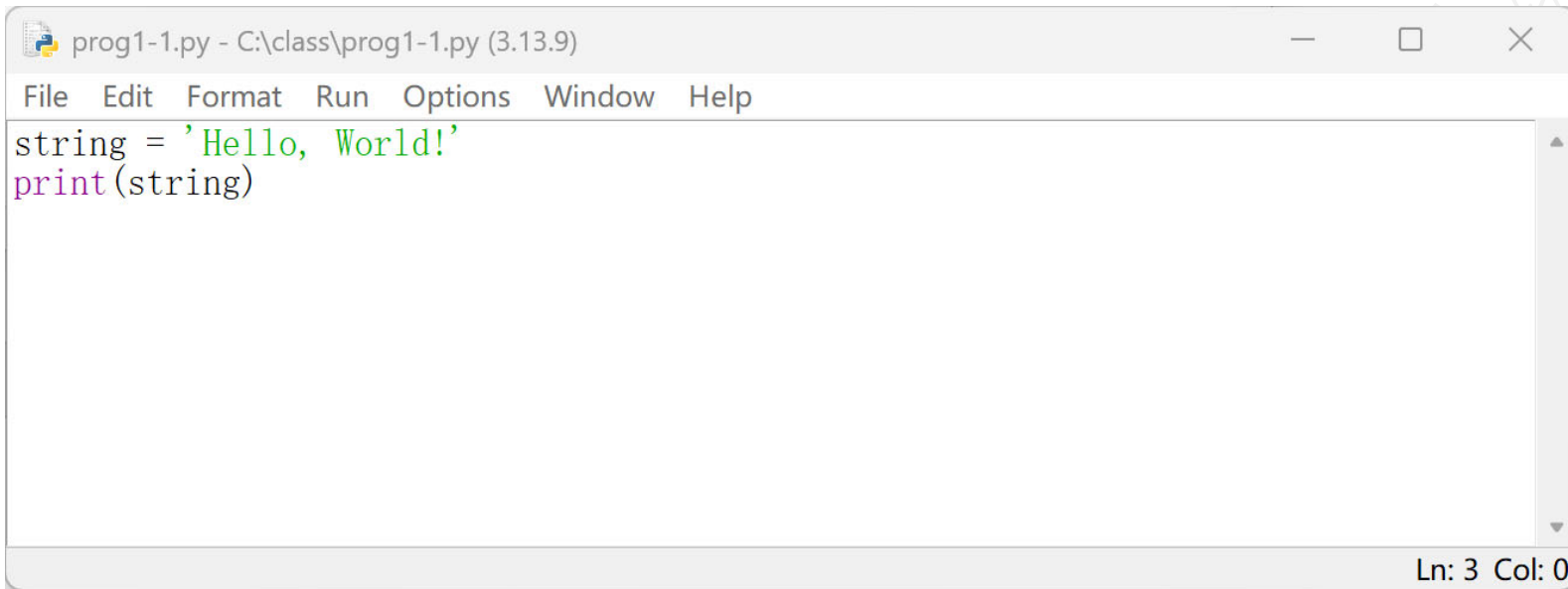
```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:51:39) [MSC v.1944 64 bit (ARM64)] on win32
Enter "help" below or click "Help" above for more information.
>>> string = 'Hello, World!'
>>> print(string)
Hello, World!
>>>
```

- Shell是交互式的解释器
- 输入一行命令，解释器就解释运行出相应结果

Python的运行方式（二）

41

文件执行方式

A screenshot of a Python IDE window titled 'prog1-1.py - C:\class\prog1-1.py (3.13.9)'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The main text area contains the following code:

```
string = 'Hello, World!'
print(string)
```

The status bar at the bottom right indicates 'Ln: 3 Col: 0'. There is a faint watermark '课程专用' (Course Special) diagonally across the right side of the image.

- 在Python的IDE环境中，创建一个以py为扩展名的文件
- 用Python解释器在Shell中运行出结果

经典 Hello World

42

```
>>> string = 'Hello, World!'
```

```
>>> print(string)
```

```
Hello, World!
```

```
>>> string
```

```
'Hello, World!'
```

```
# Filename: helloworld.py
```

```
string = 'Hello, World!'
```

```
print(string)
```

Shell方式和文件执行方式

43

- 都是**解释**运行方式
 - 整个代码段较短&无缩进变化，可选择用Shell交互方式
 - 如果代码段较长，建议使用文件执行方式

